

Detaylı Çalışma Kılavuzu

Hashing · B-Tree · Graflar · Gezme · En Kısa Yol · MST · Paradigmalar

Bu sürüm, konuyu hiç bilmeyen biri için **sıfırdan ve atlamasız** hazırlandı. Her konuda: günlük hayattan **sezgi** → resmi **tanım** → **matematiksel analiz/türetme** → **kod** → kâğıt üstünde **adım adım çözülmüş örnek** (iterasyon tablolarıyla) bulacaksın. Mor kutular için "derin/matematik" kısmıdır; istersen ilk okumada atlayıp sonra dönebilirsin.

Genişletilmiş sürüm · 2026

İçindekiler

- 1. Hashing** — Karma Tabloları
Hash fonksiyonları · yük faktörü · zincirleme · açık adresleme · olasılıksal analiz
- 2. B-Tree** — Dengeli Arama Ağaçları
Derece · arama · ekleme(bölme) · silme · yükseklik türetmesi
- 3. Graflar**: Terminoloji, Türler, Gösterimler
- 4. Graf Gezme** — BFS, DFS ve Kenar Sınıflandırması
- 5. En Kısa Yol** — Dijkstra · Bellman-Ford · Floyd-Warshall
Her biri tam iterasyon tablosuyla çözülmüş
- 6. Minimum Yayılan Ağaç** — Kruskal · Prim · Borůvka · Union-Find
- 7. Algoritma Tasarım Paradigmaları**
- 8. Hesaplamanın Sınırları ve İndirgeme (Reduction)**
Bilinen karmaşıklık sınırları · $\leq$ ilişkisi · Karatsuba · matris çarpımı indirgemesi
- 9. Genel Tekrar, Karmaşıklık Tablosu & Karar Rehberi**

1. Hashing — Karma Tabloları

SEZGI

Bir otoparkta arabayı bulmak istiyorsun. Eğer "boş bulduğun yere park et" dersin, çıkışta tüm katları gezmen gerekir — bu **O(n)** arama. Ama plakanın son rakamına göre "0 ile biten 1. kata, 1 ile biten 2. kata..." kuralı koyarsan, arabayı saniyede bulursun. İşte hash tablosu budur: bir **anahtar**ı bir formülden geçirip **doğrudan** bir adrese çevirir, böylece arama-ekleme-silme **ortalama O(1)** olur.

1.1 Önce Bir Adım Geri: Doğrudan Adresleme

En basit fikir: anahtarın kendisini dizi indeksi olarak kullanmak. Anahtarlar 0-9 arası ise, 10 elemanlık dizi açar, **A[anahtar] = değer** dersin. Hem O(1) hem dert yok. **Peki sorun ne?** Anahtarlar 0-1.000.000.000 arası ise, 1 milyar elemanlık dizi gerekir — bellek çöker. İşte hash fonksiyonu, **büyük/dağınık anahtar evrenini** küçük bir tabloya (**0..m-1**) sıkıştırır.

1.2 Hash Fonksiyonu ve İyi Bir Hash'in Özellikleri

$$h : \{ \text{tüm anahtarlar} \} \rightarrow \{ 0, 1, 2, \dots, m-1 \}$$

İyi bir hash fonksiyonunun 3 özelliği olmalı:

- Belirleyici (deterministik):** Aynı anahtar her zaman aynı kutuyu vermelidir.
- Hızlı:** O(1) zamanda hesaplanmalı (yoksa kazanç yok).
- Düzgün dağıtan (uniform):** Anahtarları kutulara olabildiğince **eşit** yaymalı ki kümelenme/çakışma azalsın.

Yaygın iki yöntem

Yöntem	Formül	Notlar
Bölme (Division)	$h(k) = k \bmod m$	Basit ve hızlı. m bir asal sayı seçilirse dağılım iyileşir; m = 2^p seçmekten kaçın (sadece son bitlere bakar).
Çarpma (Multiplication)	$h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$	0 < A < 1 sabit. m 'in asal olması gerekmez; ikinin kuvveti olabilir.

ÖRNEK

m = 10, **h(x) = x mod 10**: **h(53)=3**, **h(127)=7**, **h(23)=3**.

Son ikisi (53 ve 23) aynı kutuya (3) gitmek istiyor → **çakışma (collision)**. Çakışmalar kaçınılmazdır (**Güvercin yuvası ilkesi**: anahtar sayısı kutu sayısından fazlaysa en az iki anahtar aynı kutuyu paylaşır). Bütün mesele çakışmaları **yönetmektir**.

1.3 Yük Faktörü (Load Factor) α

$$\alpha = N / m \quad (N = \text{eleman sayısı}, m = \text{kutu sayısı})$$

α, "tablo ne kadar dolu" sorusunun cevabıdır ve performansın kalbidir. α büyüdükçe çakışma artar. α belirli bir eşiği (genelde 0.7-0.75) aşınca tablo **iki katına büyütülür** ve tüm elemanlar yeni tabloya **yeniden yerleştirilir (rehashing)**. Rehashing tek seferde O(N) maliyetlidir ama seyrek olduğu için **amortize (dağıtılmış) maliyet** yine O(1) kalır.

1.4 Çakışma Çözümü #1: Ayrık Zincirleme (Open Hashing / Separate Chaining)

TANIM

Her kutu bir **bağlı listenin başıdır**. Aynı kutuya düşen tüm elemanlar o listede zincirlenir. Arama: önce kutuyu hesapla, sonra o kutunun listesinde gez. Liste ne kadar kısaysa o kadar hızlı.

GÖRSEL ÖRNEK

m=10 'a sırayla 15, 25, 35 (hepsi **mod 10 = 5**) eklenirse 5. kutu şöyle olur:
kutu[5] → [35] → [25] → [15] → null (yeni eleman başa eklenir → O(1) ekleme).
Aramada **get(25)**: kutu 5'e git, listeyi gez, 35 değil, 25 → bulundu.

```
public class OpenHashing<K, V> {
    private static class Node<K, V> {
        K key; V value; Node<K, V> next;
        Node(K key, V value) { this.key = key; this.value = value; }
    }
    private final int m;
    private final Node<K, V>[] table;
    public OpenHashing(int m) { this.m = m; table = new Node[m]; }

    private int hash(K key) { return (key.hashCode() & 0x7fffffff) % m; }

    public void put(K key, V value) { // EKLE / GÜNCELLE
        int i = hash(key);
        for (Node<K,V> n = table[i]; n != null; n = n.next)
            if (n.key.equals(key)) { n.value = value; return; } // varsa güncelle
        Node<K,V> node = new Node<>(key, value); // yoksa başa ekle
        node.next = table[i]; table[i] = node;
    }
    public V get(K key) { // GETİR
        for (Node<K,V> n = table[hash(key)]; n != null; n = n.next)
            if (n.key.equals(key)) return n.value;
        return null;
    }
    public void delete(K key) { // SİL
        int i = hash(key);
        if (table[i] == null) return;
        if (table[i].key.equals(key)) { table[i] = table[i].next; return; }
        for (Node<K,V> n = table[i]; n.next != null; n = n.next)
            if (n.next.key.equals(key)) { n.next = n.next.next; return; }
    }
}
```

DETAY — 0X7FFFFFFF NEDEN?

Java'da `hashCode()` negatif dönebilir ve negatif sayının `%`'si negatif çıkar (dizi indeksi olamaz!). `& 0x7fffffff`, en soldaki işaret bitini sıfırlayıp sayıyı garantili pozitif yapar.

1.5 Çakışma Çözümü #2: Açık Adresleme (Closed Hashing / Open Addressing)

TANIM

Ayrı liste yok; **herkes tablonun içinde** oturur. Kutu doluysa bir **yoklama dizisi (probe sequence)** ile başka kutular denenir, ilk boş kutuya yerleşilir. Aramada da aynı diziyle ilerlenir; boş kutuya rastlanırsa "yok" denir. Burada her zaman $N \leq m$ olmalıdır (tablodan fazla eleman sığmaz).

Yöntem	Yoklama formülü $p(k,i)$	Kümelenme
Doğrusal (Linear)	$(h(k) + i) \bmod m$	Birincil kümelenme: dolu kutular uzun bloklar oluşturur.
Karesel (Quadratic)	$(h(k) + (i^2+i)/2) \bmod m$	İkincil kümelenme: aynı $h(k)$ 'li anahtarlar aynı sırayı izler ama bloklaşma azalır.
Çift Hashing (Double)	$(h_1(k) + i \cdot h_2(k)) \bmod m$	Adım boyu anahtara göre değişir → kümelenme neredeyse yok, en iyi dağılım.

$i = 0, 1, 2, \dots$ deneme numarasıdır. $i=0 \rightarrow$ ilk (asıl) kutu; dolu ise $i=1, 2, \dots$ ile devam.

ÖRNEK — DOĞRUSAL YOKLAMA (ADIM ADIM)

$m=7$, $h(k)=k \bmod 7$. Sırayla **50, 700, 76, 85, 92, 73** ekleyelim:

Anahtar	$h(k)$	Deneme dizisi	Yerleştiği kutu
50	1	kutu 1 (boş)	1
700	0	kutu 0 (boş)	0
76	6	kutu 6 (boş)	6
85	1	1 dolu $\rightarrow$ 2 (boş)	2
92	1	1 dolu $\rightarrow$ 2 dolu $\rightarrow$ 3 (boş)	3
73	3	3 dolu $\rightarrow$ 4 (boş)	4

Son tablo: [0]=700, [1]=50, [2]=85, [3]=92, [4]=73, [5]=boş, [6]=76. 73'ün 4'e itildiğine dikkat: bu "birincil kümelenmedir" — dolu blok büyüdükçe sonraki eklemeler daha çok itilir.

ÖRNEK — ÇİFT HASHİNG

$m=7$, $h_1(k)=k \bmod 7$, $h_2(k)=1+(k \bmod 5)$. $k=85$ eklerken: $h_1=1$ (dolu). Adım $h_2(85)=1+(85 \bmod 5)=1+0=1 \rightarrow i=1$: $(1+1 \cdot 1)=2 \dots$ Burada adım boyu anahtara özeldir; farklı anahtarlar farklı adımlarla ilerlediği için doğrusaldaki gibi blok oluşmaz.

```
public class LinearProbing<K, V> {
    private static class Entry<K, V> {
        K key; V value; boolean deleted;          // deleted = "mezar taşı"
        Entry(K key, V value){ this.key=key; this.value=value; }
    }
    private final int m;
    private Entry<K, V>[] table;
    private int N;
    private int hash(K key){ return (key.hashCode() & 0x7fffffff) % m; }

    public void put(K key, V value) {
        if (N == m) return;                      // tablo dolu
        int i = hash(key);
        while (table[i] != null && !table[i].deleted) {
            if (table[i].key.equals(key)) { table[i].value = value; return; }
            i = (i + 1) % m;                      // doğrusal yoklama
        }
        table[i] = new Entry<>(key, value); N++;
    }
    public V get(K key) {
        int i = hash(key);
        while (table[i] != null) {                // boş kutuya gelince dur
            if (!table[i].deleted && table[i].key.equals(key)) return table[i].value;
            i = (i + 1) % m;
        }
        return null;
    }
    public boolean delete(K key) {
        int i = hash(key);
        while (table[i] != null) {
            if (table[i].key.equals(key)) { table[i].deleted = true; N--; return true; }
            i = (i + 1) % m;
        }
        return false;
    }
}
```

TUZAK — SİLMEDE "MEZAR TAŞI" (TOMBSTONE)

Açık adreslemede bir elemanı `null` yaparak silersen **arama zinciri kırılır**: silinen kutudan sonra yerleştirilmiş elemanlar artık bulunamaz! Çözüm: kutuyu `deleted=true` ile işaretle. Arama bu kutunun **üzerinden geçer** (durmaz) ama içeriği "yok" sayar. Ekleme ise mezar taşı kutusunu yeniden kullanabilir. Çok silme/ekleme sonrası tablo mezar taşıyla dolarsa performans için rehash gerekir.

1.6 Olasılıksal Analiz — Neden "Ortalama $O(1)$ "? DERİN

BEKLENEN ÇAKIŞMA SAYISI

N anahtar m kutuya rastgele (uniform) attığımızı düşün. Herhangi iki belirli anahtarın **aynı kutuya** düşme olasılığı $1/m$ 'dir. Toplam anahtar çifti sayısı $C(N,2)=N(N-1)/2$ olduğundan, beklenen çakışma sayısı:

$$E[\text{çakışma}] = C(N,2) \cdot (1/m) = N(N-1) / (2m)$$

Buradan görülür ki çakışma sayısı N^2/m ile büyür; m büyük (yani α küçük) tutuldukça çakışma azalır.

BİR KUTUDAKİ BEKLENEN ELEMAN (ZİNCİRLEME)

Her anahtar bir kutuya $1/m$ olasılıkla düşer; bu bir **Bernoulli** denemesidir. Belirli bir kutuya düşen eleman sayısı **Binom(N , $1/m$)** dağılımına uyar, beklenen değeri:

$$E[\text{bir kutudaki eleman}] = N \cdot (1/m) = \alpha$$

Yani zincir uzunluğu ortalama α . Başarısız aramada tüm zinciri gezeriz $\rightarrow 1+\alpha$ iş; başarılı aramada ortalama yarısını $\rightarrow 1 + \alpha/2$ iş. α sabit tutulursa (örn. ≤ 0.75) bu **$O(1)$** demektir.

AÇIK ADRESLEMEDE BEKLENEN YOKLAMA SAYISI

Uniform yoklama varsayımıyla, ilk denemenin dolu çıkma olasılığı $\approx \alpha$, ikincinin $\approx \alpha^2$, ... Başarısız aramada beklenen yoklama sayısı geometrik seriyle:

E[yoklama] = $\sum_{i \geq 0} \alpha^i = 1 / (1 - \alpha)$

Başarılı aramada ise integral yaklaşımıyla:

E[başarılı] $\approx (1/\alpha) \cdot \ln(1/(1-\alpha))$

Doğrusal yoklama, kümelenme yüzünden daha kötüdür:

$\text{başarısız} \approx \frac{1}{2}(1 + 1/(1-\alpha)^2)$ $\text{başarılı} \approx \frac{1}{2}(1 + 1/(1-\alpha))$

Ana fikir: $\alpha \rightarrow 1$ 'de (tablo dolarken) bu ifadeler patlar. Bu yüzden tabloyu asla tam doldurmayız. Bu türetmelerin klasik kaynağı: Knuth, *The Art of Computer Programming*.

İşlem	Ortalama	En Kötü
Arama / Ekleme / Silme	O(1) (α sabit)	O(N) (her şey tek kutuya düşerse)
Bellek		O(N + m)

2. B-Tree — Dengeli Arama Ağaçları

SEZGİ

Sıradan bir ikili arama ağacına (BST) veriyi **sıralı** eklersen (1,2,3,4,5...) ağaç sağa doğru bir **zincire** dönüşür ve arama $O(n)$ 'e yavaşlar — yani ağaç olmanın faydası kaybolur. **Dengeli ağaçlar** bu felaketi önler. **B-Tree** ise dengeli ağacın disk/veritabanı kralıdır: her düğüme tek anahtar değil **onlarca/yüzlerce anahtar** koyar. Neden? Çünkü diskten bir blok okumak RAM'den binlerce kat yavaştır; bir hamlede çok veri okuyup ağacın **yüksekliğini** 2-3'e düşürmek, milyonlarca kaydı bile birkaç disk okumasıyla bulmak demektir.

2.1 Dengeli Ağaç Olmanın Şartları

- Her düğüm anahtarlarını **sıralı** tutar; küçükler sola, büyükler sağa giden çocuklara ayrılır.
- Tüm **yapraklar aynı seviyededir** (ağaç bir tarafa sarkmaz; mükemmel yükseklik dengesi).
- Bu sayede arama/ekleme/silme **her zaman $O(\log n)$** olur (BST'deki gibi bozulmaz).

2.2 Minimum Derece t ve Kapasiteler

Bir B-Tree, bir **minimum derece** $t \geq 2$ ile tanımlanır. t , düğümlerin kapasitesini belirler:

Özellik	En az (kök hariç)	En çok
Düğümdeki anahtar sayısı	$t - 1$	$2t - 1$
Düğümün çocuk sayısı	t	$2t$

Bir iç düğümün çocuk sayısı, anahtar sayısının **tam 1 fazlasıdır** (k anahtar varsa, aralarındaki ve uçlardaki k+1 "boşluğa" k+1 çocuk asılır). **Kök** alt sınırdan muaftır: en az 1 anahtarı yeter.

t	Min anahtar	Max anahtar	Max çocuk
2 (2-3-4 ağacı)	1	3	4
3	2	5	6
100	99	199	200

2.3 Arama (Search)

Kökten başla. Düğüm içindeki sıralı anahtarlar arasında ara: anahtar bu düğümdeyse bitti; değilse, hangi iki anahtar **arasına** düştüğüne bakıp o aralığın çocuğuna in. Yaprğa kadar bulunamazsa "yok". Her seviyede en çok $2t-1$ anahtara bakılır, yükseklik $O(\log_t n)$ olduğu için toplam **$O(\log n)$** .

2.4 Ekleme ve Düğüm Bölme (Split) — Tam Örnek

KURAL

Anahtar her zaman bir **yaprğa** eklenir. Eğer ekleneceği düğüm **doluysa** ($2t-1$ anahtar), önce o düğüm **bölünür**: ortadaki anahtar **bir üst düğüme yükselir**, kalanlar iki çocuğa paylaşılır. Bölme yukarı doğru zincirleme olabilir; kök bölünürse ağacın boyu 1 artar (tek büyüme yolu budur).

ÖRNEK — $T=3$ (DÜĞÜMDE MAX 5 ANAHTAR)

Sırayla ekleyelim: **78, 52, 81, 40, 33, 90, 35, 20, 38, 80, 100**

1) İlk 5 anahtar tek köke sığar, sıralı tutulur:

[33 40 52 78 81]

2) **90** gelir → kök dolu (5 anahtar) → **bölme**. Ortadaki **52** yukarı çıkar:

kök: [52]

sol: [33 40] sağ: [78 81 90]

(**90** zaten 52'den büyük, sağ çocuğa sıralı girer.)

3) **35** → 52'den küçük → sol çocuk [33 35 40].

4) **20** → sol çocuk [20 33 35 40].

5) **38** → sol çocuk [20 33 35 38 40] (artık 5 = dolu).

6) **80** → 52'den büyük → sağ çocuk [78 80 81 90].

7) **100** → sağ çocuğa gider ama [78 80 81 90 100] dolu olur (5).

Yeni bir ekleme bu düğümü taşıırsa ortadaki (81) köke çıkar:

kök: [52 81]

[20 33 35 38 40] [78 80] [90 100]

Gördüğün gibi ağaç **derinleşmek yerine yana genişliyor** ve hep dengeli kalıyor.

2.5 Silme (Deletion) — Genel Bakış

Silme biraz daha çetrefillidir çünkü bir düğüm **minimumun altına** ($t-1$ 'in altına) düşmemelidir. Üç temel durum:

- **Yapraktan silme, düğüm yine de dolu:** Doğrudan çıkar.
- **İç düğümden silme:** Silinen anahtarın yerine **selefi** (sol alt ağacın en büyüğü) ya da **halefi** (sağ alt ağacın en küçüğü) getirilir; sonra o yaprakтан silinir.
- **Düğüm minimuma düşerse:** Komşu kardeşten anahtar **ödünç alınır** (borrow/rotate); kardeş de minimumdaysa iki düğüm + aradaki ata anahtarı **birleştirilir (merge)**. Birleşme kökü boşaltırsa boy 1 azalır.

2.6 Yükseklik Türetmesi — Neden $O(\log n)$? DERİN

ALT SINIRDAN YÜKSEKLİK

En "seyrek dolu" (yani en yüksek) B-Tree'yi düşün: kök en az 1 anahtar, diğer her düğüm en az $t-1$ anahtar. Çocuk sayısı en az t olduğundan, seviye seviye **düğüm sayısı** en az şöyle artar:

$$\text{seviye 0: } 1, \quad \text{seviye 1: } 2, \quad \text{seviye 2: } 2t, \quad \text{seviye 3: } 2t^2, \quad \dots \quad \text{seviye } h: 2t^{h-1}$$

Toplam anahtar sayısı en az:

$$n \geq 1 + (t-1) \cdot \sum_{i=1..h} 2t^{i-1} = 1 + 2(t-1) \cdot (t^h - 1) / (t - 1) = 2t^h - 1$$

Buradan $t^h \leq (n+1)/2$, yani:

$$h \leq \log_t((n+1)/2) = O(\log_t n)$$

Yorum: t büyüdükçe (düğüm başına çok anahtar) logaritmanın tabanı büyür, yükseklik düşer. Bir milyon anahtar ve $t=100$ için yükseklik yalnızca $\sim 2-3$ 'tür $\rightarrow 2-3$ disk okumasında her şey bulunur.

B-TREE VS B+TREE

Gerçek veritabanları çoğu zaman **B+Tree** kullanır: tüm gerçek veriler yapraklarda tutulur ve yapraklar birbirine zincirlenir. Böylece "aralık sorguları" (örn. 50 ile 80 arası tüm kayıtlar) yaprakları soldan sağa tarayarak çok hızlı yapılır. İç düğümler sadece yön levhası gibidir.

3. Graflar: Terminoloji, Türler, Gösterimler

SEZGİ

Graf, "noktalar" ve onları bağlayan "çizgiler"den oluşan bir haritadır. Şehirler ve yollar, insanlar ve arkadaşlıkları, web sayfaları ve linkleri — hepsi graftır. Noktalara **köşe/düğüm (vertex)**, çizgilere **kenar (edge)** denir. Bu bölümdeki algoritmaların hepsi bu yapı üstünde çalışır.

3.1 Tanım ve Temel Terimler

$G = (V, E)$ V : köşeler kümesi, $E \subseteq V \times V$: kenarlar kümesi

Terim	Anlamı
Köşe / Kenar	Noktalar / bağlantılar. $ V $ köşe, $ E $ kenar sayısı.
Komşu (adjacent)	Aralarında kenar olan iki köşe.
Derece (degree)	Bir köşeye bağlı kenar sayısı. Yönlü grafta in-degree (gelen) ve out-degree (giden) ayrı.
Yürüyüş / İz / Yol	Walk : köşe-kenar dizisi (tekrar serbest). Trail : kenar tekrarı yok. Path (yol) : köşe tekrarı yok.
Döngü (cycle)	Başladığın köşeye geri dönen, köşe tekrarsız kapalı yol.
Bağlı (connected)	Her köşeden her köşeye bir yol varsa graf bağlıdır. Yoksa birden çok bileşene ayrılır.
Ağırlık (weight)	Kenara atanan sayı (mesafe, maliyet, süre). Ağırlıklı graf.

EL SIKIŞMA TEOREMİ

Her kenarın **iki ucu** vardır, dolayısıyla derecelere toplam **2 katkı** yapar:

$$\sum_{v \in V} \deg(v) = 2 \cdot |E|$$

Sonuç: bir grafta **tek dereceli köşe sayısı her zaman çifttir**. (Königsberg problemini çözen anahtar gözlem!)

3.2 Graf Türleri (örneklerle)

Tür	Tanım & örnek
Önemsiz (Trivial)	Tek köşe, sıfır kenar.
Boş (Null)	Köşeler var, kenar yok. $\deg=0$ her yerde.
Basit (Simple)	Ne self-loop ne çoklu kenar. Standart varsayım.
Çoklu (Multigraph)	İki köşe arası birden çok kenar olabilir; self-loop yok. (Örn. iki şehir arası 3 farklı yol.)
Yarı (Pseudo)	Hem self-loop hem çoklu kenara izin verir.
Tam (Complete) K_n	Her köşe diğer herkese bağlı. Kenar sayısı $n(n-1)/2$. K_4 = 6 kenar.
Düzenli (Regular)	Tüm köşelerin derecesi eşit. "k-düzenli": her köşe k dereceli.
İki Parçalı (Bipartite)	Köşeler iki gruba ayrılır; kenarlar yalnız gruplar arası. (Örn. öğrenciler-dersler.)
Yönsüz / Yönlü	Kenarın yönü yok / var. Yönlü: $A \rightarrow B \neq B \rightarrow A$.
Döngüsel / Döngüsüz	Döngü içeriyor / içermiyor. Yönlü + döngüsüz = DAG (görev sıralama, derleyiciler).
Sonlu / Sonsuz	Köşe-kenar sayısı sonlu mu sonsuz mu (örn. sonsuz ızgara).

3.3 Özel Yollar: Euler ve Hamilton

	Euler	Hamilton
Ne tekrarsız?	Her kenardan bir kez geç	Her köşeyi bir kez ziyaret et
Kapalı tur koşulu	Bağlı + tüm köşeler çift dereceli	Genel basit koşul yok (NP-zor problem)
Yarı-açık koşul	Tam 2 köşe tek dereceli ise Euler <i>yolu</i> (tur değil) vardır	—

TARİHSEL — KÖNİGSBERG'İN 7 KÖPRÜSÜ

1736'da Euler, 7 köprünün her birinden tam bir kez geçip başlangıca dönmenin mümkün olup olmadığını sordu. Şehir 4 kara parçası (köşe) ve 7 köprü (kenar) ile modellenir. **Dört köşenin de derecesi tekti** (3, 3, 3, 5). Euler turu için hepsi çift olmalıydı; dahası Euler *yolu* için en çok 2 tek dereceli köşe olabilirdi. 4 tane vardı → **imkânsız**. Bu kanıt, graf teorisinin doğum belgesidir.

3.4 Graf Gösterimleri (Bunu bilmek şart)

Bir grafi bilgisayarda iki temel şekilde tutarız; algoritmaların hızını doğrudan bu seçim belirler:

Gösterim	Nasıl?	Bellek	"u-v komşu mu?"	"u'nun komşuları?"
Komşuluk Matrisi (Adjacency Matrix)	$A[u][v]=1$ kenar varsa	$O(V^2)$	$O(1)$	$O(V)$
Komşuluk Listesi (Adjacency List)	Her köşenin komşu listesi	$O(V+E)$	$O(\text{derece})$	$O(\text{derece})$

HANGİSİNİ SEÇ?

Seyrek graf (az kenar, $E \ll V^2$) → **liste** (az bellek, komşu gezme hızlı). **Yoğun graf** (çok kenar, $E \approx V^2$) veya sık "komşu mu?" sorusu → **matris**. Floyd-Warshall matris kullanır; Dijkstra/Kruskal genelde liste.

4. Graf Gezme — BFS, DFS ve Kenar Sınıflandırması

SEZGİ

Bir grafi "dolaşmanın" iki temel yolu var. **BFS** (Genişlik Öncelikli) bir taşı suya attığında oluşan **dalgalar** gibi yayılır: önce 1 adım uzaktakiler, sonra 2 adım uzaktakiler... **DFS** (Derinlik Öncelikli) bir labirentte **bir koridoru sonuna kadar** gidip çıkmaza girince geri dönen kâşif gibidir. İkisi de $O(V+E)$ 'dir ama farklı işler için kullanılır.

4.1 BFS — Genişlik Öncelikli Arama (Queue ile)

Bir **kuyruk (FIFO queue)** kullanır. Başlangıç köşesini kuyruğa koy, "ziyaret edildi" işaretle. Kuyruktan bir köşe çek, ziyaret edilmemiş tüm komşularını işaretleyip kuyruğa ekle. Kuyruk boşalana dek devam et. **En önemli özelliği**: ağırlıksız bir grafta kaynaktan her köşeye **en az kenarlı (en kısa) yolu** bulur.

```
void bfs(Graph g, int source) {
    boolean[] visited = new boolean[V];
    Queue<Integer> queue = new LinkedList<>();
    visited[source] = true; queue.add(source);
    while (!queue.isEmpty()) {
        int u = queue.poll();          // kuyruktan çek
        process(u);
        for (int v : g.neighbors(u))
            if (!visited[v]) { visited[v] = true; queue.add(v); }
    }
}
```

ÖRNEK

Köşeler A(komşu: B,C), B(D), C(D,E), D, E. A'dan BFS: kuyruk başlangıç [A] → A çek, B,C ekle [B,C] → B çek, D ekle [C,D] → C çek, E ekle (D zaten var) [D,E] → D çek [E] → E çek []. **Ziyaret sırası**: A, B, C, D, E. Katmanlar: {A}=0, {B,C}=1, {D,E}=2 adım uzakta.

4.2 DFS — Derinlik Öncelikli Arama (Özyineleme / Stack)

Bulunduğu köşeden **olabildiğince derine** gider, sıkışınca geri çekilir (backtrack). Özyinelemeli yazılır ya da bir **yığın (stack)** ile. Standart DFS, her köşeye iki **zaman damgası** basar: **d[u]** = keşif anı (girişte), **f[u]** = bitiş anı (tüm işi bitince). Bu damgalar kenar türlerini belirler.

```
int time = 0;
void dfs(Graph g, int u) {
    visited[u] = true;
    d[u] = ++time;          // keşif zamanı
    for (int v : g.neighbors(u))
        if (!visited[v]) { parent[v] = u; dfs(g, v); }    // ağaç kenarı
    f[u] = ++time;          // bitiş zamanı
}
```

4.3 DFS Kenar Türleri

Tür	Ne zaman?	Anlamı
Ağaç Kenarı (Tree)	DFS ilk kez yeni bir köşeye bu kenarla ulaşır	DFS ağacını oluşturur
Geri Kenar (Back)	Bir köşeden ağaçtaki bir atasına kenar	DÖNGÜ VAR! Döngü tespitinin anahtarı
İleri Kenar (Forward)	Bir köşeden zaten torunu olan köşeye (yönlü)	Kısayol kenarı
Çapraz Kenar (Cross)	Ata-torun ilişkisi olmayan köşeler arası	Farklı dallar/alt ağaçlar arası

EN KRİTİK BİLGİ

Geri kenar bulduysan grafta döngü vardır. Yönsüz grafta DFS yaparken, atan olmayan ve daha önce ziyaret edilmiş bir komşuya rastlarsan bir döngü yakalamışsındır. Bu, döngü tespitinin en pratik yoludur.

ÖRNEK

A'dan DFS: **A→G→C→F** ve **G→D** ağaç kenarları çizilir. Şimdi **F** 'nin komşusu olarak ağaçta üstte kalan **A** çıkarsa, **F→A** bir **geri kenardır** ve **A→G→C→F→A** döngüsünü açığa çıkarır. **C** ile **D** farklı dallardaysa ve aralarında kenar varsa bu bir **çapraz kenardır**.

4.4 BFS/DFS Nerede Kullanılır?

İş	Hangisi?
Ağırlıksız grafta en kısa yol (en az kenar)	BFS
Bağlı bileşenleri bulma	İkisi de
Döngü tespiti	DFS (geri kenar)
Topolojik sıralama (DAG, görev önceliği)	DFS (bitiş zamanına göre)
Labirent/oyun çözümü, geri izleme (backtracking)	DFS

5. En Kısa Yol Algoritmaları

SEZGİ & ORTAK FİKİR: "GEVŞETME" (RELAXATION)

Üç algoritmanın da kalbinde aynı işlem var. Her köşe için kaynağa "şu ana dek bilinen en kısa mesafe" $dist[v]$ tutulur (başta kaynak 0, gerisi ∞). Bir $u \rightarrow v$ kenarına bakıldığında sorulur: "u üzerinden v'ye gitmek, şu anki $dist[v]$ 'den kısa mı?" Eğer öyleyse güncellenir:

$$\text{eğer } dist[u] + \text{ağırlık}(u,v) < dist[v] \text{ ise } dist[v] = dist[u] + \text{ağırlık}(u,v)$$

Buna **gevşetme** denir. Algoritmalar yalnızca "**hangi sırayla, kaç kez gevşetiriz**" konusunda ayrışır.

Algoritma	Ne bulur?	Negatif kenar?	Karmaşıklık	Paradigma
Dijkstra	Tek kaynak $\rightarrow$ herkes	Hayır	$O((V+E) \log V)$	Greedy
Bellman-Ford	Tek kaynak $\rightarrow$ herkes	Evet + döngü tespiti	$O(V \cdot E)$	Dinamik Prog.
Floyd-Warshall	Herkes $\rightarrow$ herkes	Evet (döngü yoksa)	$O(V^3)$	Dinamik Prog.

5.1 Dijkstra (Greedy + Min-Heap)

MANTIK

Hep "**henüz kesinleşmemiş köşeler arasından dist'i en küçük olanı**" seç (min-heap ile), kesinleştir, komşularını gevşet. Bir köşe kesinleştğinde mesafesi **asla değişmez** (pozitif ağırlık garantisi).

```
void dijkstra(Graph g, int source) {
    int[] dist = new int[V]; Arrays.fill(dist, INF); dist[source] = 0;
    MinHeap heap = new MinHeap(); heap.insert(source, 0);
    while (heap.size() > 0) {
        int u = heap.extractMin().vertex; // en yakın kesinleşmemiş köşe
        if (visited[u]) continue;
        visited[u] = true;
        for (Edge e : g.edges(u)) { // gevşetme
            int v = e.to, w = e.weight;
            if (!visited[v] && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w; heap.insert(v, dist[v]);
            }
        }
    }
}
```

TAM ÇÖZÜLMÜŞ ÖRNEK

Kenarlar (yönsüz, ağırlıklı): A-B:4, A-C:2, C-B:1, B-D:5, C-D:8, B-E:10, D-E:2. Kaynak = A. Her satır, o köşe kesinleştikten sonraki $dist[]$ durumudur (∞ = henüz ulaşılmadı):

Kesinleşen	A	B	C	D	E
başlangıç	0	∞	∞	∞	∞
A (0)	0	4	2	∞	∞
C (2)	0	3 (2+1)	2	10 (2+8)	∞
B (3)	0	3	2	8 (3+5)	13 (3+10)
D (8)	0	3	2	8	10 (8+2)
E (10)	0	3	2	8	10

Sonuç: A'dan mesafeler $\rightarrow$ B:3, C:2, D:8, E:10. (E'ye en kısa yol $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E = 2+1+5+2 = 10$; doğrudan $B \rightarrow E:10$ 'dan A üzerinden 13 değil, daha kısası bulundu.)

Avantajları	Dezavantajları
Min-heap ile çok hızlı; GPS, ağ yönlendirme için ideal.	Negatif kenarda yanlış çalışır ; negatif döngü tespit edemez.

TUZAK — NEGATİF AĞIRLIK

Dijkstra "kesinleştirdiğim köşeye daha kısa yol olamaz" varsayar. Negatif kenar varsa ileride mesafeyi düşüren bir kenar çıkabilir ve bu varsayım çöker $\rightarrow$ yanlış sonuç. Bu durumda **Bellman-Ford** kullan.

5.2 Bellman-Ford (Dinamik Programlama)

MANTIK

"En iyiyi seç" demez; **tüm kenarları V-1 kez** baştan sona gevşetir. En kısa yol en fazla V-1 kenardan oluşabileceği için V-1 tur sonunda her mesafe kesinleşir. Bu sabır, negatif kenarlarla doğru çalışmasını sağlar; ekstra bir tur daha hâlâ gevşetiyorsa **negatif döngü** vardır.

```
boolean bellmanFord(Graph g, int source) {
    int[] dist = new int[V]; Arrays.fill(dist, INF); dist[source] = 0;
    for (int i = 0; i < V - 1; i++) // V-1 tur
        for (Edge e : g.edges) { // TÜM kenarları gevşet
            if (dist[e.from] != INF && dist[e.from] + e.weight < dist[e.to])
                dist[e.to] = dist[e.from] + e.weight;
        }
    for (Edge e : g.edges) // negatif döngü kontrolü
        if (dist[e.from] != INF && dist[e.from] + e.weight < dist[e.to]) {
            System.out.println("Negatif döngü var!"); return false;
        }
    return true;
}
```

TAM ÇÖZÜLMÜŞ ÖRNEK (NEGATİF KENARLI, YÖNLÜ)

Köşeler A,B,C,D,E (yani V=5 → V-1=4 tur). Kenarlar, her turda **bu sırayla** gevşetilir:

1) D→E : 2 2) C→D : -3 3) B→C : 5 4) A→B : 4 (kaynak = A).

Kenarları bilerek "ters" sıraya koydum ki mesafenin **tur tur nasıl yayıldığını** net göresin:

Tur sonu	A	B	C	D	E	Bu turda ne oldu?
başlangıç	0	∞	∞	∞	∞	—
Tur 1	0	4	∞	∞	∞	Sadece A→B işledi (0+4)
Tur 2	0	4	9	∞	∞	B→C işledi (4+5)
Tur 3	0	4	9	6	∞	C→D işledi (9-3), negatif kenar!
Tur 4	0	4	9	6	8	D→E işledi (6+2)

Sonuç (A'dan): B:4, C:9, D:6, E:8. Mesafenin her turda **bir köşe ilerlediğine** dikkat — işte bu yüzden en kötü durumda V-1 tur gerekir. Ekstra bir tur daha çalıştırınca hiçbir mesafe değişmez → **negatif döngü yoktur**. Negatif kenarın (C→D:-3) mesafeyi düşürdüğüne dikkat; Dijkstra bunu doğru yapamazdı.

SÜPER GÜÇ

V-1 tur sonrası hâlâ gevşeyen kenar = **negatif ağırlıklı döngü** (döngü her turda maliyeti düşürür, bitmez). Yani Bellman-Ford negatif döngüyü **tespit eder** — Dijkstra'nın yapamadığı. Maliyet O(V·E).

5.3 Floyd-Warshall (Herkesten Herkese, DP)

MANTIK

Tek kaynak değil, **tüm köşe çiftleri** için en kısa yolu tek seferde bulur. Temel soru: "i 'den j 'ye giderken k köşesini **ara durak** yapsam yol kısalmı mı?" Bu soru, **her olası ara durak k** için **tüm (i,j) çiftlerine** sorulur.

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

```
void floydWarshall(int[][] dist, int V) {
    for (int k = 0; k < V; k++) // izin verilen ara durak
        for (int i = 0; i < V; i++) // başlangıç
            for (int j = 0; j < V; j++) // varış
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}
```

TAM ÇÖZÜLMÜŞ ÖRNEK

Köşeler 1,2,3,4. Yönlü kenarlar: 1→2:3, 1→4:7, 2→1:8, 2→3:2, 3→1:5, 3→4:1, 4→1:2, 4→3:6 .

Başlangıç matrisi (sıra=i, sütun=j; köşegen 0, kenar yoksa ∞):

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	6	0

k=1 (1 üzerinden): örn. 2→1→4 = 8+7=15 (∞'dan iyi) → d[2][4]=15 ; 4→1→2=2+3=5 → d[4][2]=5 .

k=2 (2 üzerinden): 1→2→3 = 3+2=5 → d[1][3]=5 ; 4→2→3=5+2=7 ama d[4][3] zaten 6, değişmez.

k=3 (3 üzerinden): 1→3→4 = 5+1=6 → d[1][4] 7'den 6'ya düşer; 2→3→4=2+1=3 → d[2][4] 15'ten 3'e.

k=4 (4 üzerinden): 2→4→1 = 3+2=5 → d[2][1] 8'den 5'e; 3→4→1=1+2=3 → d[3][1] 5'ten 3'e.

Sonuç matrisi (her hücre = en kısa mesafe):

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	6	0

Üç iç içe döngü → 0(v³) . Negatif döngü varsa köşegende negatif değer belirir (kontrol için kullanılır).

Artıları	Eksileri
Tek koşuda tüm çiftler; negatif kenarı kaldırır; kod 3 satır döngü.	0(v³) zaman, 0(v²) bellek; seyrek grafta V×Dijkstra daha hızlı olabilir.

6. Minimum Yayılan Ağaç (MST)

SEZGİ

Birkaç şehri birbirine kabloyla bağlayacaksın. Her olası kablonun bir maliyeti var. Tüm şehirleri **birbirine bağlı tutan** ama **toplam maliyeti en düşük** kablo kümesini ararsın. İşte bu **Minimum Yayılan Ağaç (Minimum Spanning Tree)**'tir. Sonuç bir "ağaçtır": döngü yoktur, çünkü döngüdeki herhangi bir kenar gereksiz maliyet demektir.

6.1 Yayılan Ağacın Özellikleri

- Grafın **tüm köşelerini** içerir (kimse dışarıda kalmaz).
- Tam olarak **V-1 kenar** içerir.
- Döngü içermez** (bir ağaçtır).

"Minimum" olanı, bu koşulları sağlayanlar arasında **kenar ağırlıkları toplamı en küçük** olandır. (Tüm kenar ağırlıkları farklıysa MST tektir; eşit ağırlıklar varsa birden çok MST olabilir.)

NEDEN GREEDY ÇALIŞIR? — KESME ÖZELLİĞİ (CUT PROPERTY)

Grafı iki gruba ayıran herhangi bir "kesme" düşün. Bu kesmeyi geçen kenarlar arasında **en hafif olan**, mutlaka bir MST'ye dahildir. Hem Kruskal hem Prim bu prensibi kullanır: her adımda güvenli "en ucuz güvenli kenarı" eklerler. İşte greedy'nin burada **kesin doğru** sonuç vermesinin matematiksel garantisi budur.

6.2 Union-Find (Birleştir-Bul) — Kruskal'ın Motoru

Kruskal'da sürekli "bu iki köşe zaten bağlı mı?" diye sorarız (bağlıysa kenar eklemek döngü yaratır). Bunu çok hızlı yapan yapı Union-Find'dir. İki optimizasyonla neredeyse-sabit zamanlı olur:

- Yol Sıkıştırma (Path Compression):** **find** sırasında gezilen düğümleri doğrudan köke bağla.
- Rütbeye Göre Birleştirme (Union by Rank):** Küçük ağacı büyüğün altına as ki ağaç sığ kalsın.

```
static int[] parent;
static int find(int i) {                // kök temsilciyi bul
    if (parent[i] == i) return i;
    return parent[i] = find(parent[i]); // yol sıkıştırma
}
static void unionSet(int x, int y) {    // iki kümeyi birleştir
    int xroot = find(x), yroot = find(y);
    parent[xroot] = yroot;
}
```

Bu işlemlerin amortize maliyeti, "ters Ackermann fonksiyonu" $\alpha(n)$ ile ifade edilir — pratikte 4'ten küçük, yani neredeyse sabit. Bu yüzden Kruskal'ın asıl maliyeti kenarları **sıralamaktan** gelir.

6.3 Kruskal Algoritması

ADIMLAR

- Tüm kenarları **maliyete göre küçükten büyüğe** sırala.
- Sıradaki en ucuz kenarı al.
- Bu kenar bir **döngü oluşturmuyorsa** (iki ucu farklı kümedeyse) ağaca ekle ve kümeleri birleştir; oluştuyorsa **atla**.
- Ağaçta **V-1** kenar olunca dur.

TAM ÇÖZÜLMÜŞ ÖRNEK

5 köşe (A,B,C,D,E) → **V-1=4** kenar seçeceğiz. Kenarlar sıralanmış halde:

Sıra	Kenar	Maliyet	Karar
1	B-C	1	Döngü yok → EKLE {B,C}
2	C-E	1	Döngü yok → EKLE {B,C,E}
3	A-B	2	Döngü yok → EKLE {A,B,C,E}
4	B-E	2	B,E aynı kümede → döngü → ATLA
5	A-C	3	A,C aynı kümede → döngü → ATLA
6	B-D	3	Döngü yok → EKLE {A,B,C,D,E} → 4 kenar, DUR

MST = { B-C, C-E, A-B, B-D }, toplam maliyet = 1+1+2+3 = 7 .

```
static void kruskalMST(int V, Edge[] edges) {
    Edge[] result = new Edge[V]; int e = 0;
    parent = new int[V];
    for (int v = 0; v < V; v++) parent[v] = v; // herkes kendi kümesinde
    Arrays.sort(edges, (a, b) -> a.weight - b.weight); // maliyete göre sırala
    int i = 0;
    while (e < V - 1 && i < edges.length) {
        Edge next = edges[i++];
        int x = find(next.src), y = find(next.dest);
        if (x != y) { result[e++] = next; unionSet(x, y); } // döngü yoksa ekle
    }
}
```

KARMAŞIKLIK TÜRETMESİ — NEDEN $O(E \log E) = O(E \log V)$? **DERİN**

Kruskal iki işten oluşur: **(1) kenarları sıralamak** ve **(2) Union-Find işlemleri**.

$$T(|E|, |V|) = |E| \cdot \log |E| \text{ (sıralama)} + |E| \cdot \alpha(|V|) \text{ (union-find)}$$

İkinci terimdeki $\alpha(|V|)$, **ters Ackermann fonksiyonudur** ve pratikte 4'ten küçüktür $\rightarrow$ neredeyse sabit. Yani $|E| \cdot \alpha(|V|) \approx |E|$, sıralama teriminin yanında yutulur:

$$T \approx |E| \cdot \log |E| + |E| = \Theta(|E| \cdot \log |E|)$$

Şimdi püf nokta: bir grfta kenar sayısı en çok $|E| \leq |V|^2$ 'dir. O hâlde:

$$\log |E| \leq \log(|V|^2) = 2 \cdot \log |V|$$

Sabit çarpan (2) Θ içinde kaybolur, dolayısıyla:

$$\Theta(|E| \cdot \log |E|) = \Theta(|E| \cdot 2 \cdot \log |V|) = \Theta(|E| \cdot \log |V|)$$

Sonuç: Kruskal'ın karmaşıklığını ister $\log |E|$ ister $\log |V|$ ile yaz — ikisi aynı kapıya çıkar. (**Ödev notu:** ters Ackermann'ın Union-Find içinde nasıl ortaya çıktığını ve Prim/Borůvka algoritmalarını ayrıca incele.)

6.4 Prim Algoritması (alternatif greedy)

Kruskal "en ucuz kenarları toplar"; Prim ise "tek bir tohumdan ağacı büyütür". Bir köşeden başla; her adımda **ağaca bitişik kenarlar** arasından en ucuzunu seç ve yeni köşeyi ağaca kat. Min-heap ile $O(E \log V)$.

ÖRNEK (YUKARIDAKİ GRAFTA, A'DAN BAŞLA)

Ağaç = {A}. Bitişik en ucuz: A-B(2) $\rightarrow$ ağaç {A,B}. Bitişikler: B-C(1), B-E(2), A-C(3)... en ucuz B-C(1) $\rightarrow$ {A,B,C}. Sonra C-E(1) $\rightarrow$ {A,B,C,E}. Son olarak B-D(3) $\rightarrow$ {A,B,C,D,E}. Aynı MST, toplam 7. Farklı yol, aynı sonuç.

6.5 Borůvka Algoritması

Her bileşenin **kendi en ucuz çıkış kenarını aynı anda** seçip birleştirir; her turda bileşen sayısı en az yarıya iner $\rightarrow O(\log V)$ tur. **Paralleleştirmeye en uygun** MST algoritmasıdır.

Algoritma	Yaklaşım	Karmaşıklık	İyi olduğu yer
Kruskal	Kenar topla (Union-Find)	$O(E \log E)$	Seyrek graf
Prim	Ağaç büyüt (min-heap)	$O(E \log V)$	Yoğun graf
Borůvka	Bileşenleri birleştir	$O(E \log V)$	Paralel işleme

KRUSKAL — ARTI / EKİ

Artıları: anlaşılması kolay; MST'yi garanti eder; bazı varyantları paralelleştirilebilir; bağlantısız graflarda her bileşen için ayrı ağaç (orman) bulur; **seyrek graflarda hızlı**.

Eksileri: yoğun graflarda (çok kenar) görece yavaş; dinamik graf güncellemelerini (kenar ekle/çıkar) iyi desteklemez.

KARIŞTIRMA!

MST $\neq$ En Kısa Yol. MST tüm grafi en ucuza *bağlar*; iki belirli nokta arası en kısa yolu vermez. Dijkstra "A'dan B'ye en kısa", Kruskal "her şeyi en ucuza bağla" sorusunu çözer.

7. Algoritma Tasarım Paradigmaları

SEZGI

Paradigma = problem çözmek için kullandığımız genel "düşünme stratejisi". Bu kılavuzdaki algoritmaların çoğu üç büyük aileden birine girer. Yeni bir problemde "hangi aileye benziyor?" sorusu, çözüme nereden başlayacağını söyler.

7.1 Böl ve Yönet (Divide and Conquer)

Problemi **bağımsız** alt parçalara böl → her parçayı (genelde özyinelemeyle) ayrı çöz → sonuçları **birleştir**.

- Örnekler:** Merge Sort, Quick Sort, ikili arama (binary search), hızlı çarpma.
- Maliyet analizi:** Genelde bir **özyineleme bağıntısı** (recurrence) ile, çoğu zaman **Master Teoremi** ile çözülür. Örn. Merge Sort:
 $T(n)=2T(n/2)+O(n) = O(n \log n)$.
- Ayırt edici:** Alt problemler **örtüşmez** (her biri bağımsız).

7.2 Dinamik Programlama (Dynamic Programming)

Problemi **üst üste binen** (tekrar eden) alt problemlere böl → her alt problemi **yalnızca bir kez** çöz, sonucu bir tabloda sakla (*memoization*) → tekrar gerekince hesaplama yerine tablodan oku.

- Örnekler:** Bellman-Ford, Floyd-Warshall, Fibonacci (memo'lu), sırt çantası (knapsack), en uzun ortak alt dizi.
- İki şart:** (1) **Örtüşen alt problemler**, (2) **Optimal alt yapı** (bütünün optimal çözümü, parçaların optimal çözümlerinden kurulur).
- Böl-yönetten farkı:** Burada alt problemler tekrar eder; DP bu tekrarı tabloyla önler.

7.3 Açgözlü Yaklaşım (Greedy)

Her adımda **o an en iyi görünen** seçimi yap, geri dönüp düzeltme yapma. Hızlıdır ama her problemde doğru sonucu garanti **etmez** — yalnızca "açgözlü seçim özelliği" ve "optimal alt yapı" olan problemlerde işler.

- Örnekler:** Dijkstra, Kruskal, Prim, Huffman kodlama, para üstü problemi (bazı para sistemlerinde).
- Dikkat:** Greedy bazı problemlerde yanlış cevap verir (örn. negatif ağırlıklı en kısa yol → Dijkstra patlar; 0/1 knapsack → greedy yanlış, DP gerekir).

ÜÇÜNÜ TEK CÜMLEDE AYIRT ET

Böl-yönet: "Parçala, ayrı ayrı çöz, birleştir." • **Dinamik programlama:** "Aynı alt problemi bir kez çöz, not al, tekrar kullan." • **Greedy:** "Şimdilik en iyiyi kap, arkana bakma."

7.4 Hesaplamanın Sınırları (kısa not)

Bu paradigmalar verimli çözülebilen (**P** sınıfı) problemler içindir. Bazı problemler ise (örn. genel Hamilton döngüsü, gezgin satıcı) bilinen hiçbir paradigmayla polinom zamanda çözülemez; bunlara **NP-zor** denir. Bu sınır farkındalığı, "bu problem için hızlı algoritma arayayım mı yoksa yaklaşık çözüme mi gideyim?" kararında önemlidir.

8. Hesaplamanın Sınırları ve İndirgeme (Reduction)

SEZGİ

Her problemin doğasında bir "hız tavanı" vardır: ne kadar akıllı olursak olalım, bir problemi belli bir sınırın altında çözemeyiz. Bu bölüm iki soruyu cevaplar: (1) Bildiğimiz problemlerin **karmaşıklık sınırları** neler? (2) Bir problemi başka bir probleme **çevirerek (indirgeyerek)** onun çözümünü ve sınırını nasıl ödünç alabiliriz?

8.1 Bilinen Karmaşıklık Sınırları

Problem / Algoritma	(En kötü) Karmaşıklık
Arama algoritmaları (sıralı veride)	$O(\log n)$
Sıralama algoritmaları (karşılaştırmalı)	$O(n \log n)$
Dijkstra (en kısa yol, pozitif)	$O((E + V) \cdot \log V)$
Floyd-Warshall (tüm çiftler)	$O(V ^3)$
Bellman-Ford (negatif kenar)	$O(V \cdot E)$
Strassen'in hızlı matris çarpımı	$O(n^{2.81})$

Karşılaştırmalı sıralamanın $O(n \log n)$ sınırı bir **alt sınırdır**: hiçbir karşılaştırmalı algoritma bundan daha hızlı sıralayamaz (kanıtı "karar ağacı" argümanıyla yapılır). Strassen ise, lise yöntemiyle $O(n^3)$ olan matris çarpımını böl-yönet ile $O(n^{2.81})$ 'e düşürür.

8.2 İndirgeme (Reduction) Nedir?

TANIM

Bir **A** problemini, bir **B** problemine **polinom zamanda çevirebiliyorsak** "A, B'ye indirgenir" deriz ve $A \leq_p B$ yazarız. Anlamı: A'yı çözmek için, A'nın girdisini hızlıca B'nin girdisine dönüştür, B'yi çöz, sonucu geri çevir. Yani **B'yi çözmeyi bilen biri, A'yı da çözebilir**.

İndirgeme, sınırları iki yönde taşır — bu, indirgemenin tüm gücüdür:

$$A \leq_p B \text{ ve } T_B(n) \in O(f(n)) \text{ ise } T_A(n) \in O(f(n))$$

$$A \leq_p B \text{ ve } T_A(n) \in \Omega(f(n)) \text{ ise } T_B(n) \in \Omega(f(n))$$

Kural	Sezgisel anlamı
Üst sınır aşağı iner: B kolaysa ($O(f)$), A da en az o kadar kolaydır.	"B'yi $f(n)$ 'de çözebiliyorsam, A'yı da $f(n)$ 'de çözerim (çünkü A'yı B'ye çeviriyorum)."
Alt sınır yukarı çıkar: A zorsa ($\Omega(f)$), B de en az o kadar zordur.	"A'nın altında $f(n)$ sınırı varsa, B daha kolay olamaz; yoksa A'yı B üzerinden daha hızlı çözerdim."

"**A problemi polinom zamanda B problemine indirgendi**" cümlesi tam olarak bu $A \leq_p B$ ilişkisini anlatır.

ÖRNEK 1 — EŞLEŞTİRME ↔ SIRALAMA

Elimizde X ve Y diye iki dizi olsun. İstenen: X'in en küçük elemanını Y'nin en küçüğüyle, X'in 2. küçüğünü Y'nin 2. küçüğüyle... eşleştirmek. Bu "sıralı eşleştirme" problemini çözmek, aslında iki diziyi de **sıralamaya** indirgenir: ikisini sırala ($O(n \log n)$), sonra hizada eşleştir ($O(n)$). Yani **Eşleştirme $\leq_p$ Sıralama** ve toplam maliyet $O(n \log n)$ olur.

ÖRNEK 2 — ÇARPMA VE KARATSUBA

n basamaklı iki sayının çarpımını ele alalım. Okulda öğrendiğimiz "her basamağı her basamakla çarp" yöntemi $O(n^2)$ maliyetlidir. **Karatsuba algoritması** ise bunu **böl ve fethet** ile hızlandırır: sayıları ikiye böler, 4 yerine **3 çarpma** ile birleştirir; özyineleme bağıntısı $T(n)=3 \cdot T(n/2)+O(n) \rightarrow O(n^{\log_2 3}) \approx O(n^{1.585})$. Böylece büyük sayı çarpımı naif $O(n^2)$ 'den daha hızlı çözülür.

ÖRNEK 3 — MATRİS ÇARPIMI ≡ MATRİS KARESİ ALMA

"İki $n \times n$ matrisi çarpmak" ile "bir matrisin karesini almak" aynı zorluktadır; biri diğerine indirgenir. $A \cdot B$ çarpımını istiyorsan, şu $2n \times 2n$ blok matrisi kurup **karesini** al:

$$M = \begin{bmatrix} 0 & A \\ 0 & 0 \end{bmatrix} \implies M^2 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \dots \rightarrow \text{blokta } A \cdot B \text{ belirir}$$

Daha düzgün kurulumla $M = \begin{bmatrix} 0 & A \\ 0 & 0 \end{bmatrix}$ ve $N = \begin{bmatrix} 0 & 0 \\ B & 0 \end{bmatrix}$ gibi bloklarla, kare alma işleminin bir bloğunda tam olarak $A \cdot B$ çıkar. Sonuç: **çarpma $\leq_p$ kare alma** (ve tersi). Yani matris karesi almak için hızlı bir algoritma bulan biri, matris çarpımını da aynı hızda çözmüş olur — sınır iki yöne de geçer.

NEDEN ÖNEMLİ?

İndirgeme, "yeni bir problemi sıfırdan çözmek" yerine "zaten çözülmüş bir probleme benzetmek" demektir. Hem pratik (çözümü hazır al) hem teoriktir: **NP-zorluk** kanıtlarının tamamı indirgemeye dayanır — bilinen zor bir problemi yeni probleme indirgeyerek "bu da en az onun kadar zor" denir.

9. Genel Tekrar, Karmaşıklık Tablosu & Karar Rehberi

9.1 Tüm Konuların Karmaşıklık Özeti

Yapı / Algoritma	Ne işe yarar	Ortalama / Tipik	En Kötü	Bellek
Hash Tablosu	Anahtarla hızlı erişim	$O(1)$	$O(N)$	$O(N+m)$
B-Tree	Sıralı, disk-dostu indeks	$O(\log n)$	$O(\log n)$	$O(n)$
BFS / DFS	Gezme, bileşen, döngü	$O(V+E)$	$O(V+E)$	$O(V)$
Dijkstra	Tek kaynak (pozitif)	$O((V+E) \log V)$	—	$O(V)$
Bellman-Ford	Tek kaynak (negatif)	$O(V \cdot E)$	$O(V \cdot E)$	$O(V)$
Floyd-Warshall	Tüm çiftler	$O(V^3)$	$O(V^3)$	$O(V^2)$
Kruskal / Prim	Minimum yayılan ağaç	$O(E \log V)$	$O(E \log V)$	$O(V+E)$

9.2 "Hangi Durumda Hangisi?" Karar Rehberi

- Bir anahtarla veriye anında ulaşmak?** → Hash tablosu (ama sıralı tarama gerekmiyorsa).
- Sıralı tutma, aralık sorgusu, disk indeksi?** → B-Tree / B+Tree.
- Ağırlıksız grafta en az adımlı yol?** → BFS.
- Döngü tespiti / topolojik sıralama?** → DFS.
- Pozitif ağırlıklı, tek kaynaktan en kısa yollar?** → Dijkstra.
- Negatif ağırlık var ya da negatif döngü mü arıyorsun?** → Bellman-Ford.
- Küçük grafta herkesten herkese mesafe?** → Floyd-Warshall.
- Her şeyi en ucuza bağlamak (şebeke kurmak)?** → Kruskal / Prim (MST).

9.3 En Sık Yapılan Hatalar

Hata	Doğrusu
Açık adreslemede elemanı null'layarak silmek	Mezar taşı (deleted) ile işaretle
Negatif kenarlı grafta Dijkstra kullanmak	Bellman-Ford'a geç
MST'yi en kısa yol sanmak	MST bağlar, en kısa yol vermez
Bölme yönteminde $m=2^p$ seçmek	Asal m seç (dağılım için)
BST'ye sıralı veri ekleyip "neden yavaş?" demek	Dengeli ağaç (B-Tree/AVL) kullan

SON SÖZ

Bu konular tek bir hikâyenin parçaları: Hash ve B-Tree **veriye nasıl sakladığın**, graf algoritmaları ise **ilişkili veriye nasıl çözümlediğin** ile ilgilidir; paradigmalar da hepsinin altındaki **düşünme biçimidir**. Her bölümün "Sezgi" kutusunu önce kavra, sonra örnekleri **kendi elinle kâğıt üstünde** tekrar çöz — özellikle Dijkstra/Bellman-Ford/Floyd tablolarını ve Kruskal kararlarını. Ezber değil, **neden öyle çalıştığını** anlamak kalıcı olandır.